Numerical evaluation of the DIS structure functions in the C++ code

October 20, 2026

Abstract

This note describes how the C++ code in `cpp/` evaluates the leading-order inclusive DIS structure functions with a BK-evolved dipole: the optical-theorem result (OT) and the finite-energy (FE) constrained results with the dipole evolved to x_B (4D) or to $x_{\mathbb{P}}$ (5D). It explains the integration variables, the three ways the $M^2_{q\bar{q}}$ integral of the 5D cross section can be done, and the choice of integration routine, and it compares the results with the Python reference implementation in `src/small_x_physics`. The C++ code computes the same integrals as the Python code. With the default settings it agrees with the Python results within their statistical errors, and it reaches comparable precision 30–80 times faster on 8 cores.

Contents

1 The quantities **1**

2 Building blocks **2**

3 Integration variables **3**

4 The $M^2_{q\bar{q}}$ integral of FE 5D **3**
4.1 Original form (`original`) 3
4.2 Integration by parts (`by-parts`) 3
4.3 Filon rule (`filon`) 4
4.4 Automatic choice (`auto`, the default) 4

5 Integration routines **4**

6 Comparison with the Python reference **5**

7 Performance **6**

8 Recommendations and caveats **7**

1 The quantities

The code computes, for one quark flavour of mass m and charge Z_f, the structure functions

$$F_\lambda = \frac{Q^2}{4\pi^2\alpha_{\rm em}}\,\sigma_\lambda, \qquad \lambda = L, T, \qquad F_2 = F_L + F_T, \tag{1}$$

in three ways, exactly as `test_bk_integration.py`. With $r^2 = |\mathbf{u}-\mathbf{u}'|^2 = u^2 + u'^2 - 2uu'\cos\theta$, $W^2 = Q^2(1/x_B - 1)$ and $\epsilon^2 = Q^2 z(1-z) + m^2$:

OT (`BK_OT_CS.py`), with the dipole at $Y = \ln(x_0/x_B)$:

$$\sigma_\lambda^{\mathrm{OT}} = \int_{r_{\min}}^{r_{\max}} \mathrm{d}r \int_{z_{\min}}^{1-z_{\min}} \mathrm{d}z \, \frac{\sigma_0}{2} \, \frac{1}{4\pi} \, \frac{2\pi r}{z(1-z)} \, |\psi_\lambda(r,z)|^2 \, 2[1 - S_Y(r)]. \tag{2}$$

FE 4D (`BK_FE_4D_evolve_in_xB.py`), with the dipole at $Y = \ln(x_0/x_B)$:

$$\sigma_\lambda^{\mathrm{4D}} = \int \mathrm{d}u \, \mathrm{d}u' \, \mathrm{d}\theta \, \mathrm{d}z \; P_\lambda(u,u',\theta,z) \; T_Y(u,u',\theta,z) \; \frac{\zeta J_1(\zeta)}{2\pi r^2}, \qquad \zeta = r\sqrt{z(1-z)W^2 - m^2}. \tag{3}$$

FE 5D (`BK_FE_5D_evolve_in_xP.py`), with the dipole at $Y(M_{q\bar{q}}^2) = \ln(x_0/x_{\mathbb{P}})$ and $x_{\mathbb{P}} = (M_{q\bar{q}}^2 + Q^2)/(W^2 + Q^2)$:

$$\sigma_\lambda^{\mathrm{5D}} = \int \mathrm{d}u \, \mathrm{d}u' \, \mathrm{d}\theta \, \mathrm{d}z \int_{4m^2}^{W^2} \mathrm{d}M_{q\bar{q}}^2 \; P_\lambda \; T_{Y(M_{q\bar{q}}^2)} \; \Theta(k^2) \; \frac{z(1-z)}{4\pi} \, J_0(kr), \qquad k^2 = z(1-z)M_{q\bar{q}}^2 - m^2. \tag{4}$$

Here $P_\lambda = \frac{\sigma_0}{2} \frac{1}{4\pi} \frac{2\pi u u'}{z(1-z)} |\psi_\lambda(u,u',\theta,z)|^2$ collects the light-cone wavefunction and its conjugate, and

$$T_Y = 1 - S_Y(u) - S_Y(u') + S_{4,Y}(u,u',\theta,z) \tag{5}$$

is the target amplitude, built from the BK dipole S_Y and the Gaussian quadrupole $S_{4,Y}$ at finite or large N_c. The limits are those of the Python script: $r, u, u' \in [10^{-6}, 10]$ GeV^{-1}, $z \in [10^{-8}, 1 - 10^{-8}]$, $\theta \in [0, 2\pi]$.

All numbers in this note are for $Q^2 = 2$ GeV^2, $x_B = 10^{-3}$, $m = 0.14$ GeV, $Z_f^2 = 2/3$, $\sigma_0 = 2 \cdot 2.57 \cdot 18.81$ GeV^{-2}, $x_0 = 0.01$, the BK solution with MV initial condition in `data/mv.dat`, and the finite-N_c quadrupole unless stated otherwise.

2 Building blocks

All of these agree with the Python implementation point by point. The unit tests compare them at about 2000 points each, generated by `tests/reference/generate_reference.py`.

BK dipole. The Python code uses scipy's `RectBivariateSpline` with $s = 0$. That is the unique bicubic tensor-product interpolant with not-a-knot end conditions, so the C++ code rebuilds it from the data by 1D not-a-knot splines along each axis, stored as one polynomial per grid cell. Arguments outside the grid are clamped, N is clipped to $[0, 1]$ and $S = \max(1 - N, 10^{-14})$, as in Python. Agreement: 10^{-13} in N. The interpolation is a template parameter, and a bilinear alternative is provided.

Large-x extrapolation. Optionally (`--large-x-power` p), the dipole at $x > x_0$ is $N(r,x) = N(r,x_0)\,[(1-x)/(1-x_0)]^p$ instead of the Python behaviour of keeping it at x_0. Only FE 5D evaluates the dipole at $x > x_0$. There is no Python counterpart.

Wavefunctions. The Bessel functions K_0, K_1, J_0, J_1 come from Boost.Math, or from GSL with `-Duse_boost=OFF`. Agreement: 10^{-12} relative.

Quadrupole. Dominguez et al. eq. B21 from the six pair dipoles, including the $\Delta = 0$ branch and the same 10^{-14} floor in the logarithms. Tested with an analytic GBW dipole and with the BK dipole. Agreement: 10^{-10}.

Integrands. The OT, FE 4D and FE 5D integrands agree to 10^{-9}, relative to the integrand's scale.

L and T share everything except the wavefunction, so each integrand returns both, and one integration computes them as two components. Computing them in separate integrations, as Python does, gave 10–15% smaller FE errors at equal points per polarization but cost twice as much, so at equal time the combined integration is better.

3 Integration variables

The C++ code integrates over the same domains as Python, but by default in different variables. `--variables python` restores the Python variables.

Relative coordinates (FE). The phase-space kernel of (3) is sharply peaked at $\mathbf{u} = \mathbf{u}'$, with a width of about $1/\sqrt{z(1-z)W^2}$, roughly 0.05 GeV^{-1} here. Away from the peak it falls only like $r^{-3/2}$, oscillating. Its integral converges, but its square does not: *in the Python variables* (u, u', θ) *the integrand has infinite variance* near the diagonal $u = u'$, $\theta = 0$. That is a diagonal of the integration box, which an adaptive Monte Carlo cannot follow. Section 6 shows the consequences: estimates drift upwards with the number of points, and the errors vegas reports are too small.

The C++ code integrates instead over u, the separation $r = |\mathbf{u} - \mathbf{u}'|$ on a logarithmic scale, and the angle ϕ of $\mathbf{u} - \mathbf{u}'$ relative to $\mathbf{u}$. Since $\mathrm{d}^2u\,\mathrm{d}^2u' = \mathrm{d}^2u\,\mathrm{d}^2r$, the integration measure changes as

$$\mathrm{d}u\,\mathrm{d}u'\,\mathrm{d}\theta\; uu' = \mathrm{d}u\,\mathrm{d}r\,\mathrm{d}\phi\; ur, \tag{6}$$

so the integrand gains a factor r/u'. The original domain $u' \in [r_{\min}, r_{\max}]$ is kept exactly by setting the integrand to zero outside it. The disc $r < 10^{-10}$ GeV^{-1} is left out of the logarithmic scale; it changes the result by a relative $\sim (10^{-10}\,\mathrm{GeV}^{-1} \cdot M)^2 \sim 10^{-15}$. r^2 is computed directly from r, not from $u^2 + u'^2 - 2uu'\cos\theta$, which would cancel catastrophically at $r \ll u$.

Symmetries. All integrands are invariant under $z \to 1 - z$, which swaps quark and antiquark, and under $\theta \to -\theta$, a reflection. So z runs over $[z_{\min}, 1/2]$ and the angle over $[0, \pi]$, and the result is multiplied by 4 (by 2 for OT).

Logarithmic sizes. r (OT) and u (FE) are sampled on a logarithmic scale.

With Vegas at 10^6 points, these variables make the FE 4D errors 1.8–1.9 times smaller than in the Python variables. Over repeated seeds the reported errors match the observed scatter (ratio 0.6–1.5 over 8 seeds). For OT the variables make no difference with Vegas; see Section 5.

4 The $M^2_{q\bar{q}}$ integral of FE 5D

At fixed u, u', θ, z, only T depends on $M^2_{q\bar{q}}$ in (4), through $Y(M^2_{q\bar{q}})$. With $s = k^2$, $\mathrm{d}s = z(1-z)\,\mathrm{d}M^2_{q\bar{q}}$, and $K^2 = z(1-z)W^2 - m^2$, the inner integral is

$$I = \frac{1}{4\pi}\int_0^{K^2} \mathrm{d}s\; J_0(r\sqrt{s})\, T(Y(s)). \tag{7}$$

J_0 oscillates over the whole range (up to $Kr \sim 400$ here), and the narrow peak of the 4D kernel builds up only by cancellation. That makes the original form hard for any integrator. Three forms are available (`--fe5d-method`).

4.1 Original form (`original`)

The 5D integral (4) as written, as in Python. It is used with `--variables python` and for cross-checks.

4.2 Integration by parts (`by-parts`)

Using $\frac{\mathrm{d}}{\mathrm{d}s}\left[2\sqrt{s}\,J_1(r\sqrt{s})/r\right] = J_0(r\sqrt{s})$,

$$I = \frac{KJ_1(Kr)}{2\pi r}\,T(Y_W) + \frac{1}{4\pi}\int \mathrm{d}M^2_{q\bar{q}}\,\frac{2kJ_1(kr)}{r}\,\frac{1}{M^2_{q\bar{q}} + Q^2}\,\frac{\mathrm{d}T}{\mathrm{d}Y}, \qquad Y_W = Y(W^2) = \ln x_0. \tag{8}$$

The first term is the FE 4D integrand (3) with the dipole at $x_{\mathbb{P}} = 1$. The second, the correction, vanishes wherever the dipole does not depend on Y. For the Python behaviour (dipole kept at x_0 for $x > x_0$), that is all of $x_{\mathbb{P}} > x_0$, so the correction only covers $M^2_{q\bar{q}} \lesssim 18$ GeV^2 instead of up to $W^2 \approx 2000$ GeV^2. There it barely oscillates. $\mathrm{d}T/\mathrm{d}Y$ is a central finite difference with step 10^{-4}, kept inside the dipole's rapidity range. The derivation is in `doc/fe5d_by_parts.pdf`. With Vegas this reduces the FE 5D error 12–16 times at equal points.

With the large-x extrapolation the dipole depends on Y all the way to $x_{\mathbb{P}} = 1$. Then the boundary term vanishes ($N = 0$ at $x_{\mathbb{P}} = 1$, so $T = 0$), and the correction spans the whole range and oscillates again. In that case Cuhre's error estimate fails on it: results were off by about 20 times the estimate. The code warns if this combination is requested.

4.3 Filon rule (`filon`)

The inner integral (7) is computed at each point (u, u', θ, z), so that the remaining integral is a smooth 4D one.

1. The rapidity range $[Y(K^2), Y(0)]$ is split at the dipole's *breakpoints*, where $\mathrm{d}T/\mathrm{d}Y$ may jump: the ends of the tabulated range and, with the extrapolation, $Y = 0$. On a piece where the dipole does not depend on Y, T is constant and the integral of J_0 is exact.

2. On the other pieces, T is evaluated at 8 Chebyshev–Lobatto nodes and interpolated with the barycentric formula. The nodes are in Y for $x_{\mathbb{P}} < x_0$, where BK evolution is smooth in Y, and in $x_{\mathbb{P}}$ for $x_{\mathbb{P}} > x_0$, where the extrapolation is a smooth function of x. In Y it would be stretched exponentially: the first version, with nodes in Y, was off by up to 10^{-3} of the inner integral in typical configurations, and by 5% where the integral nearly cancels.

3. Each piece is divided into 8 panels. On each panel the interpolant is replaced by a cubic in s through four points, and $J_0(r\sqrt{s})$ times the cubic is integrated exactly. This needs the moments $\int s^n J_0(r\sqrt{s})\,\mathrm{d}s$, $n \le 3$, whose antiderivative is, with $x = r\sqrt{s}$,

$$A_n(s) = s^{n+1} \sum_{j\ge 0} \frac{(-x^2/4)^j}{(j!)^2(n+j+1)} \quad (x < 4), \qquad A_n(s) = \frac{2\,I_{2n+1}(x)}{r^{2n+2}} \quad (x \ge 4), \tag{9}$$

   where $I_m(x) = \int_0^x y^m J_0(y)\,\mathrm{d}y$ follows from $I_1 = xJ_1$ and $I_m = x^m J_1 + (m-1)x^{m-1}J_0 - (m-1)^2 I_{m-2}$. The series avoids the cancellation of the recurrence at small x.

The accuracy is that of interpolating T, independently of how much J_0 oscillates. A unit test compares the Filon integrand with a brute-force 1D quadrature of (7) for the frozen and the extrapolated dipole. Each point costs about 30 FE 4D evaluations.

4.4 Automatic choice (`auto`, the default)

`by-parts` if the dipole stops depending on Y inside the $M^2_{q\bar{q}}$ range (the Python behaviour), `filon` if it depends on Y up to $x_{\mathbb{P}} = 1$ (the extrapolation). For a frozen dipole the Filon result keeps the sharp $r \to 0$ peak created by T jumping to a nonzero constant at W^2, so Cuhre needs millions of the expensive points (12.8 s against 1.5 s by parts). With the extrapolation, T vanishes smoothly at $x_{\mathbb{P}} = 1$, and Cuhre converges in about 10^5 points.

5 Integration routines

`--integrator` selects **Cuhre** (the default) or **Vegas** from Cuba 4.2.2.

Cuhre is deterministic adaptive cubature. It subdivides the region until the estimated error is below `--epsrel` (default 10^{-3}) times the result. Its error estimate compares cubature rules

of different degree. It is not statistical, so it cannot be checked by changing the seed; it was checked by repeating with a ten times smaller `--epsrel`, and the change was always within the estimate. Cuhre keeps all subregions in memory and slows down beyond a few 10^7 evaluations.

Vegas is adaptive Monte Carlo, run like Python's vegas: 10 warm-up iterations of $N/10$ points that train the grid and are discarded, then 20 iterations of N points (`--mcpoints` N). Cuba implements classic Vegas, without the stratified sampling of Python's vegas package.

Table 1 compares the Cuba routines at the same budget of 2.1×10^7 evaluations, the budget of Vegas at $N = 10^6$, with the default variables and the frozen dipole.

	OT	FE 4D	FE 5D correction	time vs. Vegas
Vegas	1.6×10^{-5}	1.8×10^{-4}	5.9×10^{-6}	1
Suave, $n_{\text{new}} = 10^4$	biased (6σ)	1.6×10^{-4}	biased (5–8σ)	≈ 1.3
Suave, $n_{\text{new}} = 10^5$	2.7×10^{-6}	1.7×10^{-4}	4.8×10^{-6}	≈ 1.3
Divonne	5.6×10^{-8}, FT off 3σ	8.0×10^{-5}	—	25–450
Cuhre	converged	9.5×10^{-6}	2.4×10^{-7}	≈ 1

Table 1: Errors of F_L from the Cuba routines at 2.1×10^7 evaluations. "Biased" means off from the reference by the stated multiple of the routine's own error.

OT. Classic Vegas cannot match the stratified sampling of Python's vegas for this smooth 2D integral: at equal points its error is 50–70 times larger, and no change of variables helped. Cuhre gives the integral to 10^{-9} in 0.1 s.

FE 5D with the extrapolation. This is the case that matters most for future work. Table 2 compares Cuhre with Suave for it, at a target relative error of 10^{-3}. The reference is Cuhre with the Filon rule at 16 nodes and 32 panels and $\epsilon_{\text{rel}} = 10^{-5}$: $F_L = 0.07582522(76)$, $F_T = 0.2611083(26)$.

	F_L off by	F_T off by	1 core	8 cores
Cuhre, Filon (default)	0.0σ	1.2σ	0.8 s	0.27 s
Suave $n_{\text{new}} = 10^4$, Filon	5–12σ	6–7σ	28–40 s	—
Suave $n_{\text{new}} = 10^5$, Filon	0.5–1.2σ	0.3–0.5σ	23 s	5.4 s
Suave $n_{\text{new}} = 10^4$, by parts	up to 2.3σ	3–4σ	25 s	—
Suave $n_{\text{new}} = 10^5$, by parts	0.4–1.8σ	1.2–1.3σ	13–15 s	5.3 s

Table 2: FE 5D with the large-x extrapolation ($p = 4$), target relative error 10^{-3}, two seeds for Suave. At 10^{-4}, Cuhre took 4.0 s on one core and was within 0.1σ of the reference.

Suave is 15–50 times slower than Cuhre, and with its usual subdivision size it is biased by several standard deviations without warning. Divonne is far too slow. Cuhre with the Filon rule is therefore the recommended method.

On 8 cores, Cuhre gains a factor of 3 for the expensive Filon integrand. For the cheap FE 4D and frozen-dipole integrands it is about 1.3 times faster on one core than with 8 worker processes (`--cores 0`).

6 Comparison with the Python reference

The Python references are vegas runs of the Python code with 10^7 points on 8 cores. For OT there is also a nested-quadrature value, accurate to 10^{-10}. For FE 5D the reference is the mean of

three runs with different seeds, with its error taken from their scatter. That scatter is about twice the error vegas reports for one run, a consequence of the infinite variance described in Section 3. Python's FE results also drift with the number of points: FE 4D F_T is 0.38246(45) at 10^6 points and 0.38347(12) at 10^7, 2.2σ apart.

	F_L	pull	F_T	pull
OT				
Python, nested quadrature	0.0994504423	—	0.3913902122	—
Python, vegas 10^7	0.09945045(3)		0.3913905(2)	
C++ Cuhre, $\epsilon_{\rm rel} = 10^{-4}$	0.0994505(70)		0.391391(38)	
C++ Cuhre, $\epsilon_{\rm rel} = 10^{-7}$	agrees with the quadrature to 5×10^{-9} relative			
FE 4D				
Python, vegas 10^7	0.099269(44)	—	0.38347(12)	—
C++ Cuhre, $\epsilon_{\rm rel} = 10^{-4}$	0.0993117(99)	+0.9	0.383456(38)	−0.1
C++ Vegas 10^6, default variables	0.09931(18)	+0.2	0.38393(54)	+0.8
C++ Vegas 10^6, Python variables	0.09910(32)	−0.5	0.3830(10)	−0.5
FE 5D				
Python, vegas, mean of three 10^7 runs	0.07606(47)	—	0.2769(11)	—
C++ Cuhre, $\epsilon_{\rm rel} = 10^{-4}$ (by parts)	0.0763908(64)	+0.7	0.276422(25)	−0.4
C++ Vegas 10^6, default variables	0.07650(12)	+0.9	0.27705(38)	+0.1
C++ Vegas 10^6, Python variables	0.0801(19)	+2.0	0.2875(43)	+2.4
Large N_c, against Python vegas 10^6				
FE 4D, C++ Cuhre 10^{-3}	0.099308(99)	−0.1	0.38352(38)	−0.6
FE 5D, C++ Cuhre 10^{-3}	0.076406(64)	+0.7	0.27639(25)	+0.6

Table 3: Structure functions at $Q^2 = 2$ GeV^2, $x_B = 10^{-3}$. The pull is the difference from the Python value over the combined error.

Table 3 shows that every C++ method agrees with Python within its errors. The largest pulls, about 2σ, are for FE 5D in the Python variables, which share the infinite-variance problem of the Python code itself. With the default settings all pulls are below 1σ.

The C++ Monte Carlo errors are reliable: over 8 seeds the scatter of the results was 0.6–1.5 times the reported error. From 10^6 to 10^7 points the default-variable Vegas results moved by at most 2σ, about 0.1% for FE 5D, compared with the 2.2σ drift of Python's FE 4D.

The extrapolated dipole has no Python reference. Three independent methods agree for it: Cuhre with the Filon rule ($\epsilon_{\rm rel} = 10^{-4}$: $F_L = 0.0758259(75)$, $F_T = 0.261111(26)$), Vegas by parts (0.075761(82), 0.26119(23)), and Vegas on the original 5D form in the Python variables, i.e. the Python method (0.07501(58), 0.2594(13)). The pulls are at most 1.4σ. The extrapolation lowers FE 5D compared with the frozen dipole: F_L by 0.7% and F_T by 5.5%.

7 Performance

Table 4 compares wall-clock times at matched precision. For the C++ code, Cuhre's `--epsrel` was set to the relative error of the Python result, and the time includes reading the BK file. The machine is an Apple M2 with 4 performance and 4 efficiency cores.

These numbers favour Python, because its reported FE errors are too small. For many kinematic points, independent C++ runs in parallel give another factor of about 4.7 on this machine: 8 concurrent FE 4D runs took 0.18 s per calculation instead of 0.86 s. The gains come from compiled code (about 2–3 times per point), Boost Bessel functions (1.8 times), computing L and T together (2 times), the integration variables and the by-parts or Filon forms, and, above all for OT and FE 5D, from replacing Monte Carlo by cubature.

Precision of	Python, 1 / 8 cores		C++	speedup, 1 / 8 cores	
OT, Python 10^6 (2×10^{-6})	41.0 s	10.6 s	0.01 s	∼4000	∼1000
FE 4D, Python 10^6 (1.2×10^{-3})	16.1 s	9.1 s	0.86 s	19	11
FE 5D, Python 10^6 (10^{-2})	25.8 s	11.9 s	0.24 s	110	50
all three	83 s	32 s	1.1 s	**75**	**30**
OT, Python 10^7 (3×10^{-7})	—	105 s	0.01 s	—	∼10000
FE 4D, Python 10^7 (3×10^{-4})	—	71 s	2.6 s	—	27
FE 5D, Python 10^7 (3×10^{-3})	—	95 s	0.62 s	—	150
all three	—	271 s	3.3 s	—	**80**

Table 4: Time to comparable precision (relative error in parentheses), frozen dipole. The C++ times are for one Cuhre process.

8 Recommendations and caveats

- The defaults (Cuhre, `--epsrel` 10^{-3}, automatic FE 5D method) give about 10^{-3} relative precision in about a second per structure function. For final numbers use `--epsrel` 10^{-4}, and check a point with a ten times smaller `--epsrel`.
- With the Filon rule below `--epsrel` 10^{-4}, raise `--filon-nodes` and `--filon-panels`; at the defaults (8, 8) the interpolation error is far below the 10^{-4} integration error.
- All checks are at $x_B = 10^{-3}$. At smaller x_B the FE kernel peak is narrower and J_0 oscillates more; spot-check a production point there against Vegas or a tighter `--epsrel`.
- Interpolating T in $x_{\mathbb{P}}$ above x_0 suits extrapolations written as functions of x. A different extrapolation that is not smooth in x would need checking with more Filon nodes.
- For cross-checks, the Python method is reproduced by the options `--integrator vegas`, `--variables python` and `--fe5d-method original`.